

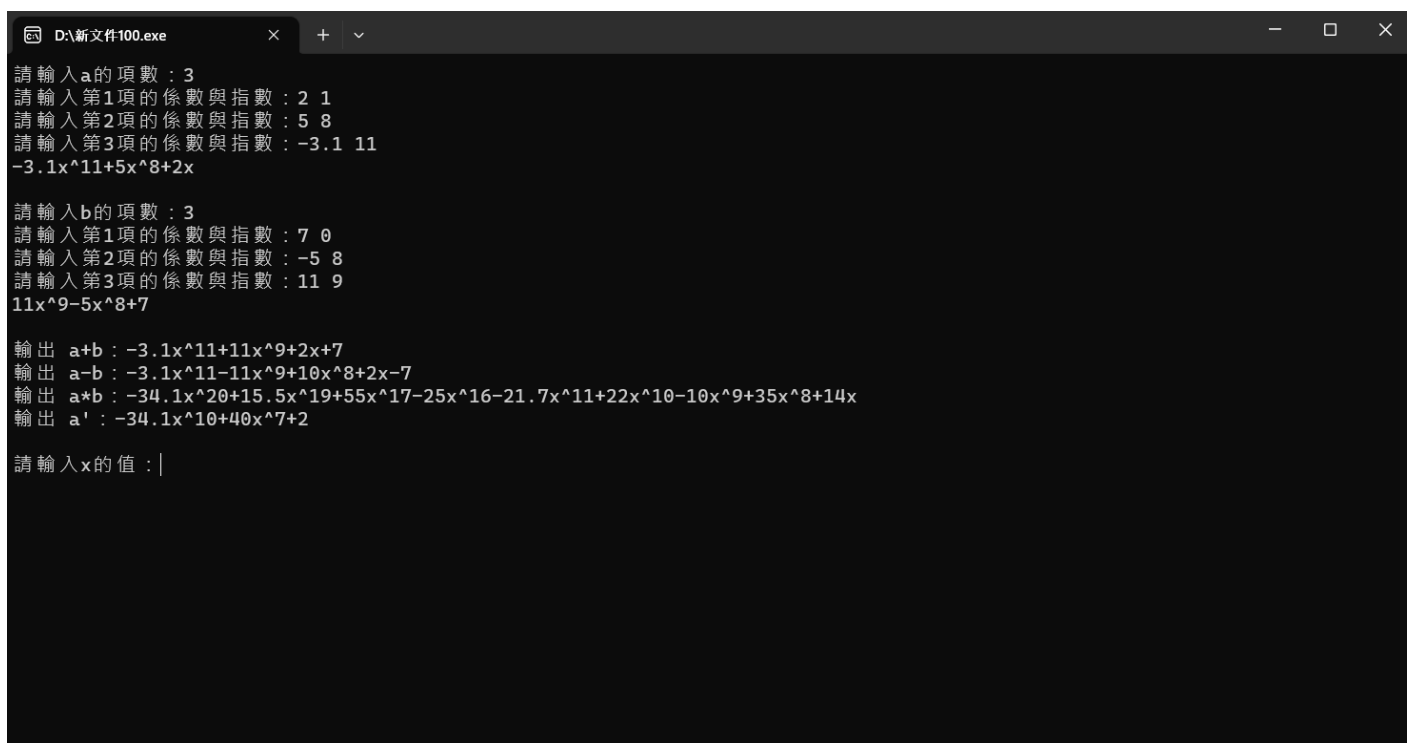
Project 1 实验报告

学号：22336049 姓名：陳日康

1、程序功能简要说明。

利用 c/c++ 设计一个一元稀疏多项式计算器，程序功能为输入并建立多项式，然后输出多项式，形式为 $n, c_1, e_1, c_2, e_2, \dots, c_n, e_n$ ，其中 n 是多项式的项数， c_i 和 e_i 分别是第 i 项的系数和指数，序列按降幂排序排列，分别计算并输出多项式相加、相减，计算多项式在 x 处的值，算出多项式 a 的导函数 a' 还有求出多项式的乘积。

2、程序运行截图，包括计算功能演示、部分实际运行结果展示、命令行或交互式界面效果等。



```
D:\新文件100.exe
請輸入a的項數：3
請輸入第1項的係數與指數：2 1
請輸入第2項的係數與指數：5 8
請輸入第3項的係數與指數：-3.1 11
-3.1x^11+5x^8+2x

請輸入b的項數：3
請輸入第1項的係數與指數：7 0
請輸入第2項的係數與指數：-5 8
請輸入第3項的係數與指數：11 9
11x^9-5x^8+7

輸出 a+b：-3.1x^11+11x^9+2x+7
輸出 a-b：-3.1x^11-11x^9+10x^8+2x-7
輸出 a*b：-34.1x^20+15.5x^19+55x^17-25x^16-21.7x^11+22x^10-10x^9+35x^8+14x
輸出 a'：-34.1x^10+40x^7+2

請輸入x的值：|
```

上图为部分运行结果，当输入了多项式 a 和 b 的数据后，程序会输出 $a+b$ ， $a-b$ ， $a*b$ 和 a 的导数 a' ，测试数据为 $a = 2x + 5x^8 - 3.1x^{11}$ ， $b = 7 - 5x^8 + 11x^9$ 。

```
D:\新文件100.exe
請輸入a的項數：3
請輸入第1項的係數與指數：2 1
請輸入第2項的係數與指數：5 8
請輸入第3項的係數與指數：-3.1 11
-3.1x^11+5x^8+2x

請輸入b的項數：3
請輸入第1項的係數與指數：7 0
請輸入第2項的係數與指數：-5 8
請輸入第3項的係數與指數：11 9
11x^9-5x^8+7

輸出 a+b：-3.1x^11+11x^9+2x+7
輸出 a-b：-3.1x^11-11x^9+10x^8+2x-7
輸出 a*b：-34.1x^20+15.5x^19+55x^17-25x^16-21.7x^11+22x^10-10x^9+35x^8+14x
輸出 a'：-34.1x^10+40x^7+2

請輸入x的值：1
輸出a的多項式的值為：3.90
-----
Process exited after 42.86 seconds with return value 0
請按任意鍵繼續 . . . |
```

上图为代码的所有运行结果，测试数据为 $a = 2x + 5x^8 - 3.1x^{11}$ ， $b = 7 - 5x^8 + 11x^9$ ，

以下为计算结果：

$$a+b = -3.1x^{11} + 11x^9 + 2x + 7;$$

$$a-b = -3.1x^{11} - 11x^9 - 10x^8 + 2x - 7;$$

$$a*b = -34.1x^{20} + 15.5x^{19} + 55x^{17} - 25x^{16} - 21.7x^{11} + 22x^{10} - 10x^9 + 35x^8 + 14x;$$

$$a' = -34.1x^{10} + 40x^7 + 2;$$

$$a(1) = 2 + 5 - 3.1$$

$$= 3.9;$$

经对比，所有的计算结果与程序运行结果一致。

3、部分关键代码及其说明。

```
7 typedef struct Polynomial {           //多項式
8     float coef;                       //係數
9     int expn;                          //指數
10    struct Polynomial *next;           //指針
11 } Polynomial, *Polyn;
12
```

这段代码定义了一个名为 'Polynomial' 的结构体，用于表示多项式的每一项。结构体包括了系数 ('coef')、指数 ('expn') 以及指向下一项的指针 ('next')。

```

13 //創建一個頭指針為head，項數為m的一元多項式
14 Polyn CreatPolyn (Polyn head, int m) {
15     head = (Polyn)malloc(sizeof(struct Polynomial));
16     head -> next = NULL;
17     for (int i=1; i<=m; i++) {
18         Polyn p = (Polyn)malloc(sizeof(struct Polynomial));
19         printf ("請輸入第%d項的係數與指數：", i);
20         scanf ("%f%d", &p->coef, &p->expn);
21         if (p->coef == 0)
22             free (p);
23         else {
24             Polyn q1, q2;
25             q1 = head;
26             q2 = head -> next;
27             while (q2 != NULL && p->expn < q2->expn) {
28                 q1 = q2;
29                 q2 = q2->next;
30             }
31             if (q2 != NULL && p->expn == q2->expn) {
32                 q2->coef += p->coef;
33                 if (q2->coef == 0) {
34                     q1->next = q2->next;
35                     free (q2);
36                 }
37                 free (p);
38             }
39             else {
40                 p -> next = q2;
41                 q1 -> next = p;
42             }
43         }
44     }
45     return head;
46 }

```

这个函数用于创建一个多项式，并根据用户输入的项数和系数指数，构建多项式链表。它会按照指数升序的方式将项插入到链表中，同时合并同类项。

```

48 void printPollyn (Polyn head) {
49     Polyn q = head -> next;
50     int f = 0;
51     if (!q) {
52         puts ("0");
53         puts ("\n");
54         return;
55     }
56     while (q) {
57         if (q->coef > 0 && f == 1) {
58             printf("+");
59         }
60         f = 1;
61         if (q->coef != 1 && q->coef != -1) {
62             printf ("%g", q->coef);
63             if (q->expn == 1)
64                 printf ("x");
65             else if (q->expn != 0)
66                 printf ("x^%d", q->expn);
67         }
68         else {
69             if (q->coef == 1) {
70                 if (q->expn == 0)
71                     printf ("1");
72                 else if (q->expn == 1)
73                     printf ("x");
74                 else
75                     printf ("x^%d", q->expn);
76             }
77             if (q->coef == -1) {
78                 if (q->expn == 0)
79                     printf ("-1");
80                 else if (q->expn == 1)
81                     printf ("-x");
82                 else
83                     printf ("-x^%d", q->expn);
84             }
85         }
86         q = q->next;
87     }
88     printf ("\n");
89 }

```

打印多项式 `printPollyn` 函数：这个函数用于比较两个多项式的大小，根据指数的大小进行比较。返回值为 1 表示第一个多项式大于第二个，-1 表示第一个小于第二个，0 表示相等。

```

91 int compare (Polyn a, Polyn b) { //比較兩個多項式的大小
92     if (a&&b) { //多項式a和b均不為空
93         if (a->expn > b->expn)
94             return 1; //a的指數大於b的指數
95         else if (a->expn < b->expn)
96             return -1;
97         else
98             return 0;
99     }
100     else if (!a&&b)
101         return -1; //a為空，b不為空
102     else if (a&&!b)
103         return 1; //b為空，a不為空
104     else if (!a&&!b)
105         return 0; //a,b均為空
106 }

```

多项式比较 `compare` 函数：这个函数用于比较两个多项式的大小，根据指数的大小进行比较。返回值为 1 表示第一个多项式大于第二个，-1 表示第一个小于第二个，0 表示相等。

```

108 Polyn addPolyn (Polyn a, Polyn b) { //求解a+b，並返回頭結點head
109     Polyn head, qc;
110     Polyn qa = a->next;
111     Polyn qb = b->next;
112     Polyn hc = (Polyn)malloc(sizeof(Polynomial));
113     hc->next = NULL;
114     head = hc;
115     while (qa || qb) {
116         qc = (Polyn)malloc(sizeof(Polynomial));
117         if (compare(qa, qb) == 1) {
118             qc->coef = qa->coef;
119             qc->expn = qa->expn;
120             qa = qa->next;
121         }
122         else if (compare(qa, qb) == 0) { //指數相同，直接相加
123             qc->coef = qa->coef + qb->coef;
124             qc->expn = qa->expn;
125             qa = qa->next;
126             qb = qb->next;
127         }
128         else {
129             qc->coef = qb->coef;
130             qc->expn = qb->expn;
131             qb = qb->next;
132         }
133         if (qc->coef != 0) { //將該節點插入鏈表中
134             qc->next = hc->next;
135             hc->next = qc;
136             hc = qc;
137         }
138         else free(qc);
139     }
140     return head;
141 }

```

上图是多项式相加 `addPolyn` 函数：这个函数用于计算两个多项式的和，并返回一个新的多项式链表。

```

143 Polyn subPolyn (Polyn a, Polyn b) { //求解a-b
144     Polyn h = b;
145     Polyn p = b->next;
146     while (p) {
147         p->coef *= -1;
148         p = p->next;
149     }
150     Polyn head = addPolyn (a, h);
151     for (Polyn i = h->next; i != 0; i = i->next) {
152         i->coef *= -1;
153     }
154     return head;
155 }

```

多项式相减 `subPolyn` 函数：这个函数用于计算两个多项式的差，并返回一个新的多项式链表。首先将第二个多项式的所有系数取相反数，然后再调用 `addPolyn` 函数实现相减操作。

```

157 double value (Polyn head, int x) { //計算x的值
158     double sum = 0;
159     for (Polyn p = head->next; p != 0; p = p->next) {
160         int tmp = 1;
161         int expn = p->expn;
162         while (expn != 0) { //指數不為0
163             if (expn < 0)
164                 tmp /= x, expn++;
165             else if (expn > 0)
166                 tmp *= x, expn--;
167         }
168         sum += p->coef*tmp;
169     }
170     return sum;
171 }

```

计算多项式值 `value` 函数：这个函数用于计算多项式在给定整数 `x` 处的值。它通过遍历多项式链表，计算每一项的值并累加得到最终结果。

```

173 Polyn derivative (Polyn head) { //計算導數
174     Polyn derivativeHead, p, q;
175     derivativeHead = (Polyn)malloc(sizeof(Polynomial));
176     derivativeHead->next = NULL;
177     p = head->next;
178     while (p!=NULL && q->expn!=0) {
179         q = (Polyn)malloc(sizeof(Polynomial));
180         q->coef = p->coef * p->expn;
181         q->expn = p->expn - 1;
182         q->next = NULL;
183         Polyn prev = derivativeHead; //尋找插入點，按升冪順序
184         Polyn current = derivativeHead->next;
185         while (current != NULL && current->expn > q->expn) {
186             prev = current;
187             current = current->next;
188         }
189         if (current != NULL && current->expn == q->expn) { //插入節點
190             current->coef += q->coef;
191             free(q);
192         }
193         else {
194             q->next = current;
195             prev->next = q;
196         }
197         p = p->next;
198     }
199     return derivativeHead;
200 }

```

计算多项式导数 `derivative` 函数：这个函数用于计算多项式的导数，并返回一个新的多项式链表。导数的计算包括计算每一项的系数和指数，按降冪排序并构建新的多项式。

```

189 Polyn multiplyPolyn (Polyn a, Polyn b) { //计算a*b
190     Polyn resultHead, pa, pb, pr, temp, node;
191     resultHead = (Polyn)malloc(sizeof(Polynomial));
192     resultHead->next = NULL;
193     pa = a->next;
194     while (pa) {
195         pb = b->next;
196         pr = resultHead;
197         while (pb) {
198             temp = (Polyn)malloc(sizeof(Polynomial));
199             temp->coef = pa->coef * pb->coef;
200             temp->expn = pa->expn + pb->expn;
201             temp->next = NULL;
202             node = resultHead;
203             while (node->next != NULL && node->next->expn > temp->expn) {
204                 node = node->next;
205             }
206
207             if (node->next != NULL && node->next->expn == temp->expn) {
208                 node->next->coef += temp->coef;
209                 free (temp);
210             }
211             else {
212                 temp->next = node->next;
213                 node->next = temp;
214             }
215             pb = pb->next;
216         }
217         pa = pa->next;
218     }
219     return resultHead;
220 }

```

计算多项式乘积 `multiplyPolyn` 函数： `Polyn multiplyPolyn (Polyn a, Polyn b)`，这个函数用于计算两个多项式的乘积，并返回一个新的多项式链表。它实现了多项式乘法的逻辑，遍历两个多项式的每一项，将它们相乘并合并同类项。

4、程序运行方式简要说明。

程序是一个多项式操作程序，可以执行多项式的创建、加减法、求导、计算多项式的值以及多项式的乘法操作。以下是程序的运行方式简要说明：

- (1) 运行程序后，首先会要求输入多项式'a'的项数(m)，然后要求输入'a'的每一项的系数和指数。这些输入会创建多项式 a 并打印出 a。同理创建 b 跟打印出 b。
- (2) 然后程序分别计算并打印出多项式 a+b 跟 a-b。
- (3) 程序还会计算并打印出多项式 a 的导数。
- (4) 跟着程序会要求输入一个整数 x 的值，程序会计算并打印出多项式 a 在 x 处的值。
- (5) 最后，程序会计算并打印出多项式 a 和 b 的乘积(a*b)。

这样就可以通过输入多项式的系数和指数以及整数 x 的值，执行上述多项式的操作。